Разработка ядра библиотеки обнаружения аномалий OmniAD: самоописание API, потоковое обучение, сериализация моделей и инфраструктура тестирования

Отчёт о научно-исследовательской работе за семестр

17 октября 2026 г.

Цель работы

Разработать и проверить ключевые подсистемы ядра библиотеки обнаружения аномалий OmniAD: самоописание API, поддержку потокового обучения, единообразную обработку ошибок и сериализацию моделей, — а также создать многоуровневую инфраструктуру автоматизированного тестирования. На основе полученного состояния библиотеки подготовить уточнённую архитектурную спецификацию бенчмарков и реализовать базовый механизм воспроизводимого сравнения адаптеров OmniAD с непосредственным использованием исходных библиотек.

Аннотация

Алгоритмы обнаружения аномалий доступны в нескольких библиотеках машинного обучения, однако их интерфейсы и соглашения о данных различаются. В scikit-learn, PyTorch, HuggingFace Transformers и River используются разные процедуры обучения и предсказания, форматы входных объектов, способы сохранения моделей и интерпретации оценок аномальности. В приложениях, объединяющих такие алгоритмы, значительная часть кода приходится на адаптацию и согласование компонентов. Это затрудняет расширение системы, диагностику ошибок и проверку корректности результата.

OmniAD разрабатывается как Python-библиотека с единым интерфейсом для алгоритмов обнаружения аномалий на табличных данных, временных рядах, текстах и изображениях. В рамках работы предусматривается развитие нескольких подсистем её ядра. Механизмы самоописания должны позволять получать сведения об алгоритмах и поддерживаемых возможностях без обучения модели. Для потоковых данных необходимо согласовать пакетный и инкрементальный жизненные циклы, включая обновление порога аномальности. Для взаимодействия с внешними вычислительными библиотеками требуется унифицированная обработка исключений, а для сохранения моделей — разделение состояния адаптера и состояния бэкенда без дублирования тяжёлых объектов.

Отдельная часть работы посвящена автоматизированным проверкам: тестированию контрактов адаптеров, совпадения результатов с нативными реализациями там, где оно определено, сериализации, потокового обучения, визуализации и собранного Python-пакета. Затем планируется пересмотреть рабочий документ проекта технической спецификации новой системы бенчмаркинга и оформить спецификацию экспериментальных измерений. На её основе будет разработан исполняемый механизм бенчмаркинга: независимая эталонная реализация, описание сценария, повторные замеры и сохранение результатов для построения отчёта. Планируемый итог семестра — проверенные подсистемы ядра и воспроизводимый сквозной бенчмарк на примере IsolationForest. Величина накладных расходов OmniAD заранее не предполагается и должна определяться экспериментально.

Практическая значимость

Разработка направлена на повышение пригодности OmniAD к использованию и сопровождению:

- Самоописание API упрощает выбор алгоритма, проверку параметров и использование библиотеки из IDE и интерактивных сред.

- Потоковое обучение позволяет применять алгоритмы с поэлементным обновлением модели без накопления всей истории наблюдений.
- Унифицированная сериализация помогает сохранять и восстанавливать составные модели и избегать повторного включения тяжёлых объектов в архив.
- Контрактные и интеграционные тесты позволяют обнаруживать несовпадения между поведением OmniAD и поддерживаемых бэкендов, а проверка wheel выявляет ошибки упаковки.
- Система бенчмаркинга даст возможность проверять сопоставимость выходных результатов и измерять накладные расходы интерфейса на фиксированных сценариях.

Задачи работы

1. Обеспечение прозрачности API библиотеки (discoverability)

1.1. Создание общего механизма интроспекции параметров адаптеров для runtime-API и генератора .pyi-стабов.

1.2. Реализация функций omniad.describe() и omniad.list_detectors() с фильтрацией по домену и возможностям алгоритмов.

1.3. Введение декларативного реестра возможностей CAPABILITY_REGISTRY и определения capabilities по интерфейсам-миксинам.

1.4. Унификация текстового и HTML-представления объектов детекторов (__repr__, __str__, _repr_html_).

1.5. Реализация определения возможностей составных адаптеров TF-IDF и BERT в зависимости от внутреннего детектора.

2. Реализация поддержки потокового (streaming) обучения

2.1. Создание единого интерфейса ThresholdStrategy для пакетной калибровки порога и инкрементального обновления.

2.2. Реализация стратегий quantile, sigma3, iqr, ewma, streaming_quantile и регистрации пользовательских стратегий.

2.3. Добавление partial_fit() и интерфейса IncrementalLearningMixin в базовый класс детектора.

2.4. Реализация правила валидации единичного наблюдения require_single_row.

2.5. Создание BaseRiverAdapter для объединения потокового и пакетного режимов обучения.

2.6. Добавление адаптера HalfSpaceTreesAdapter на основе River.

2.7. Выделение опциональных зависимостей streaming и проверка обработки отсутствующего River.

3. Унификация обработки исключений на границе с внешними библиотеками

3.1. Приведение predict_score() к общему шаблону с преобразованием ошибок бэкенда в исключения OmniAD.

3.2. Защита дополнительных публичных методов predict_map(), predict_expected() и get_feature_importances().

3.3. Аудит модуля миксинов и устранение ошибок в импортах, проверках capabilities и типах исключений.

4. Переработка механизма сериализации моделей

4.1. Разделение лёгкого состояния адаптера и состояния вычислительного бэкенда в существующем ZIP-контейнере.

4.2. Введение декларативного списка _serialization_exclude, собираемого по иерархии наследования.

4.3. Исключение дублирования состояний PyTorch, Transformers, вложенных детекторов и TF-IDF-векторизатора.

4.4. Уточнение поведения save_weights и сохранения состояния потоковых стратегий порога.

5. Построение комплексной автоматизированной системы тестирования

5.1. Разделение тестов по уровням: ядро, базовые шаблоны адаптеров, алгоритмы, общие контракты, утилиты, реестр и фабрика.

5.2. Проверка жизненного цикла BaseDetector и калибровки порогов аномальности.

5.3. Разработка parity-тестов для сопоставления скоров адаптеров с нативными реализациями.

5.4. Тестирование декларативной валидации данных, метрик и регистров пользовательских расширений.

5.5. Тестирование потокового обучения и стратегий порогов, включая продолжение обучения после загрузки модели.

5.6. Проверка заявленных возможностей алгоритмов и механизма композиции адаптеров.

5.7. Тестирование базовых адаптеров scikit-learn, PyTorch, River и Transformers.

5.8. Проверка трансляции исключений на границе внешних библиотек.

5.9. Тестирование сохранения и восстановления моделей без изменения результатов инференса.

5.10. Проверка реестра, фабрики, пресетов и опциональных зависимостей.

5.11. Тестирование утилит оконной обработки временных рядов и вспомогательных преобразований.

5.12. Организация проверок форматирования, статической типизации и запуска тестов в CI.

5.13. Добавление smoke-теста сборки wheel с запуском вне исходного репозитория.

5.14. Разработка тестов визуализации, включая headless-рендеринг и доменные форматы данных.

6. Разработка архитектурной спецификации подсистемы бенчмаркинга

6.1. Разбор существующих скриптов измерений и рабочего черновика проекта технической спецификации новой системы бенчмаркинга; выделение спорных и незакрытых решений.

6.2. Определение контракта независимой нативной реализации (native baseline) и проверки сопоставимости результатов.

6.3. Уточнение подготовки датасетов для разных доменов, структуры DatasetBundle и протоколов train/test.

6.4. Определение реестров алгоритмов и датасетов, описания сценария, формата и версионирования JSON-результата.

6.5. Описание методики повторных измерений throughput и latency, фиксирования окружения и хранения сырых наблюдений.

6.6. Согласование проверяемого объёма первой реализации и оформление очередной редакции спецификации.

7. Реализация подсистемы бенчмаркинга согласно уточнённой спецификации

7.1. Реализация DatasetBundle и синхронизированных реестров алгоритмов и датасетов; описание конфигураций и идентификаторов сценариев.

7.2. Подготовка независимого native baseline для IsolationForest и автоматической проверки эквивалентности скоров.

7.3. Реализация исполнения сценария, обработки ошибок и сохранения метаданных и сырых наблюдений в JSON.

7.4. Реализация повторных парных CPU-измерений throughput и latency с прогревом и контролем порядка запусков.

7.5. Добавление консольного интерфейса запуска и формирования CSV/Markdown-отчётов на основе исходных записей.

7.6. smoke-тесты подсистемы, запуск сквозного сценария IsolationForest, анализ результата и исправление выявленных ошибок.

7.7. При наличии времени: расширение покрытия алгоритмов, отдельные измерения памяти, возобновление прерванных прогонов и сравнение результатов разных запусков.

Список источников и научно-технических материалов

1. Tan, S. C., Ting, K. M., Liu, F. T. Fast Anomaly Detection for Streaming Data // IJCAI, 2011. P. 1511–1516.

Полный текст / материал: https://www.ijcai.org/Proceedings/11/Papers/254.pdf

Краткий обзор (ИИ): Авторы описывают Streaming Half-Space Trees: ансамбль случайных деревьев для последовательного обнаружения аномалий. Структура дерева строится без обучающих данных, а информация о заполненности узлов обновляется по окнам потока. В статье приведены алгоритмы построения дерева и потокового обновления, оценки вычислительных затрат и эксперименты на наборах данных с разным поведением потока.

Зачем это в моей работе: Источник нужен мне для объяснения того, какой алгоритм стоит за HalfSpaceTreesAdapter. Корректность самого адаптера я проверяю сравнением с реализацией River, а не повторением всех экспериментов из статьи.

2. Montiel, J., Halford, M., Mastelini, S. M., et al. River: Machine Learning for Streaming Data in Python // JMLR, 22(110), 2021. P. 1–8.

Полный текст / материал: https://www.jmlr.org/papers/v22/20-1380.html

Краткий обзор (ИИ): Статья представляет библиотеку River, объединяющую инструменты онлайн-обучения: модели, потоковую статистику, преобразования данных и средства оценки. Основные операции выполняются для отдельных наблюдений, что отличает River от пакетного API scikit-learn. Этот принцип определяет устройство адаптера для потоковых алгоритмов.

Зачем это в моей работе: Мне здесь важен интерфейс learn_one()/score_one(). Я использую эти примитивы для partial_fit() и для реализации batch fit() в BaseRiverAdapter.

3. Zaharia, M., Chen, A., Davidson, A., et al. Accelerating the Machine Learning Lifecycle with MLflow // IEEE Data Engineering Bulletin, 41(4), 2018. P. 39–45.

Полный текст / материал: https://people.eecs.berkeley.edu/~matei/papers/2018/ieee_mlflow.pdf

Краткий обзор (ИИ): Работа посвящена инфраструктуре жизненного цикла ML-моделей: фиксации параметров экспериментов, сохранению артефактов, развёртыванию и повторному использованию моделей. Для проекта важен общий принцип явного описания и хранения модельных артефактов. При этом статья не предлагает конкретную схему ZIP-контейнера OmniAD.

Зачем это в моей работе: Этот источник помогает обосновать разделение метаданных, лёгкого состояния обёртки и нативных весов. Сам формат архива и список исключаемых атрибутов я проектирую для OmniAD отдельно.

4. Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P. F. Producing Wrong Data Without Doing Anything Obviously Wrong! // ASPLOS, 2009. P. 265–276.

Полный текст / материал:
https://users.cs.northwestern.edu/~robby/courses/322-2013-spring/mytkowicz-wrong-data.pdf

Краткий обзор (ИИ): Исследование показывает, как результат измерения времени выполнения может зависеть от окружения и порядка проведения опыта, даже если сравниваемые программы работают корректно. Авторы рассматривают систематические искажения, связанные с экспериментальной процедурой, и способы их обнаружения.

Зачем это в моей работе: При сравнении накладных расходов OmniAD разница может быть небольшой. Источник показывает, что одного замера недостаточно, а потому нужно фиксировать порядок запусков и сохранять отдельные наблюдения.

5. Kalibera, T., Jones, R. E. Rigorous Benchmarking in Reasonable Time // ISMM, 2013. P. 63–74.

Полный текст / материал: https://kar.kent.ac.uk/33611/

Краткий обзор (ИИ): Авторы разбирают, как организовывать эксперименты по производительности, когда отдельные запуски дают различающиеся результаты. Рассматриваются уровни повторений, выбор объёма измерений и оценка статистической неопределённости без чрезмерного увеличения времени эксперимента.

Зачем это в моей работе: Этот источник нужен для пересмотра системы бенчмарков: следует осмысленно выбирать число повторений, а не ставить его произвольно. Полный статистический аппарат статьи в первую версию я включать не планирую.

6. Han, S., Hu, X., Huang, H., Jiang, M., Zhao, Y. ADBench: Anomaly Detection Benchmark // NeurIPS, 2022.

Полный текст / материал: https://arxiv.org/pdf/2206.09426

Краткий обзор (ИИ): ADBench рассматривает сравнение методов обнаружения аномалий в едином экспериментальном протоколе с большим набором датасетов. Основной акцент сделан на измерении качества поиска аномалий и влиянии условий задачи на результат. Для OmniAD это пример организации воспроизводимых экспериментов и корректного обращения с данными разных типов.

Зачем это в моей работе: Я не собираюсь переносить ADBench целиком. Он нужен скорее как ориентир для подготовки датасетов и будущих quality-бенчмарков -- замеры overhead OmniAD требуют своего протокола.

7. Lones, M. A. Avoiding Common Machine Learning Pitfalls // Patterns, 5(10), 2024. Article 101046.

Полный текст / материал: https://pmc.ncbi.nlm.nih.gov/articles/PMC11573893/

Краткий обзор (ИИ): Работа систематизирует распространённые ошибки ML-исследований: утечку данных, неверную процедуру оценки, ненадёжное сравнение моделей и недостаточную воспроизводимость. Отдельно подчёркивается важность согласования подготовки выборок с постановкой задачи.

Зачем это в моей работе: В OmniAD есть несколько доменов с разными правилами деления train/test. Мне важно не случайно перемешать временной ряд и не изменить условия оригинального датасета только ради единого loader API.

Обзор литературы, документации и существующих решений

Обзор охватывает библиотеки обнаружения аномалий, способы интеграции ML-моделей и материалы по проверке корректности и производительности. Используются научные публикации, официальная документация и журналы изменений. Для проектов, которые существенно развивались, отдельно рассматриваются их исходные публикации и возможности на октябрь 2026 года. В этом разделе описаны существующие решения; сопоставление их с OmniAD и выводы приведены далее.

Библиотеки обнаружения аномалий

PyOD первоначально разрабатывался как библиотека обнаружения выбросов в многомерных данных. В статье 2019 года [8] описаны общий интерфейс различных детекторов, подход к расширению набора алгоритмов и инженерные практики: автоматизированные тесты, непрерывная интеграция, контроль покрытия и сопровождаемости кода. Статья рассматривает PyOD своего времени и не описывает современную мультидоменную архитектуру.

Мультидоменное расширение PyOD произошло значительно позже. В журнале изменений [9] указано, что 6 апреля 2026 года в версии 2.1.0 добавлены EmbeddingOD и MultiModalOD для композиции кодировщиков с детекторами, прежде всего при работе с текстами и изображениями. 9 апреля появилась группа специализированных детекторов временных рядов, 11 апреля — графовые детекторы. Выпуск PyOD 3.0 от 12 апреля закрепил архитектуру из классического API, слоя ADEngine и агентного исследовательского сценария. 4 июня версия 3.6.0 добавила обработку аудио. Современная документация перечисляет шесть типов данных: табличные, временные ряды, графы, тексты, изображения и аудио [9].

В PyOD общий интерфейс детектора связан с соглашениями scikit-learn. Для мультимодальных задач используется композиция: EmbeddingOD сначала преобразует данные с помощью кодировщика, затем применяет детектор к векторному представлению. ADEngine содержит сведения об алгоритмах и средства организации последовательности действий — от описания данных до построения результатов [9]. Документация отдельно оговаривает требования к зависимостям и ограничения некоторых алгоритмов.

PyGOD предназначен для обнаружения аномалий в графах. Его авторы описывают модульные графовые алгоритмы, подготовку данных в графовом представлении и инструменты для работы с большими графами [10]. Препринт проекта появился в 2022 году, а статья в JMLR опубликована в 2024 году. PyGOD является самостоятельным проектом; появление графовых алгоритмов внутри PyOD в апреле 2026 года не следует смешивать с датой появления PyGOD.

Merlion объединяет методы обнаружения аномалий и прогнозирования временных рядов [11]. В библиотеке предусмотрены подготовка данных, постобработка и калибровка оценок, модели-ансамбли, настройка гиперпараметров и средства оценки в сценариях, имитирующих эксплуатацию модели и повторное обучение. Работа имеет выраженную специализацию на временных рядах.

Anomalib ориентирован на визуальные аномалии [12]. Он предоставляет набор методов, компоненты для создания новых моделей, инструменты обучения и оценки, визуализацию и способы экспорта. Помимо решения о наличии аномалии, для визуальных данных важны локализация дефекта и карты аномальности. Эти результаты требуют интерфейсов, отличающихся от скалярного скора для табличного наблюдения.

Интерфейсы, потоковые модели и сохранение состояния

В scikit-learn базовый интерфейс estimator включает fit(), операции доступа к параметрам get_params() и set_params(), а также соглашения о создании новых оценщиков и их совместимости

с инструментами экосистемы [16]. Для библиотек, объединяющих готовые модели, этот интерфейс служит широко используемым примером стандартизации программного контракта.

Работа Tan, Ting и Liu [1] описывает алгоритм Streaming Half-Space Trees. Он оценивает аномальность поступающих наблюдений с использованием деревьев, для которых меняются статистики заполненности областей пространства. Библиотека River [2] строится на поэлементных операциях, таких как learn_one() и score_one(), и предоставляет потоковые статистики. В документации конкретной реализации Half-Space Trees отмечены ограничения и предположения о диапазоне признаков [17].

Материал о MLflow [3] рассматривает управление жизненным циклом ML-моделей и их артефактами. Официальная документация scikit-learn [13] описывает форматы сохранения, включая pickle и joblib, а также ограничения по безопасности и совместимости версий. Документация PyOD [14] описывает сохранение модели вместе с версиями зависимостей и процедуру загрузки с явным подтверждением доверия к артефакту.

Экспериментальная проверка и оценка производительности

ADBench [6] представляет инфраструктуру оценки качества алгоритмов обнаружения аномалий на наборах данных и протоколах сравнения. Он ориентирован прежде всего на качество обнаружения; измерение дополнительных затрат программного интерфейса требует отдельной постановки эксперимента. Работа Lones [7] посвящена распространённым ошибкам ML-исследований, включая утечки данных, некорректную оценку моделей и недостаточный контроль воспроизводимости.

Исследование Mytkowicz и соавторов [4] показывает зависимость результатов измерений производительности от условий запуска и экспериментальной процедуры. Kalibera и Jones [5] рассматривают повторные измерения, неоднородность источников вариации и оценку статистической неопределённости. Оба материала относятся к методике получения и интерпретации измерений, а не к алгоритмам обнаружения аномалий.

Дополнительные источники и документация

8. Zhao Y., Nasrullah Z., Li Z. PyOD: A Python Toolbox for Scalable Outlier Detection // JMLR, 20(96), 2019. P. 1–7.

Полный текст / документация: https://www.jmlr.org/papers/volume20/19-011/19-011.pdf

Краткий обзор (ИИ): Статья представляет ранний PyOD как инструментарий обнаружения выбросов в многомерных данных с общим API, набором моделей и практиками сопровождения кода.

Зачем это в моей работе: Нужен как исходная точка: отделить идеи, уже существовавшие в 2019 году, от расширений PyOD в 2026 году.

9. PyOD. Журнал изменений и документация проекта (состояние на октябрь 2026 года).

Полный текст / документация: https://github.com/yzhao062/pyod/blob/master/CHANGES.txt ; https://pyod.readthedocs.io/en/latest/

Краткий обзор (ИИ): Журнал фиксирует версии и даты появления мультимодальных средств, временных рядов, графовых детекторов и трёхуровневой архитектуры PyOD 3. Документация описывает их актуальные интерфейсы.

Зачем это в моей работе: Позволяет не переносить возможности современного PyOD на версию 2019 года и корректно учитывать развитие конкурирующего проекта.

10. Liu K., Dou Y., Ding X. et al. PyGOD: A Python Library for Graph Outlier Detection // JMLR, 25(141), 2024. P. 1–9.

Полный текст / документация: https://www.jmlr.org/papers/volume25/23-0963/23-0963.pdf

Краткий обзор (ИИ): Описана специализированная графовая библиотека с модульными алгоритмами и средствами подготовки, запуска и проверки моделей.

Зачем это в моей работе: Нужна для изучения требований графового домена, даже если поддержка графов не входит в текущие задачи OmniAD.

11. Bhatnagar A. et al. Merlion: A Machine Learning Library for Time Series. 2021.

Полный текст / документация: https://arxiv.org/pdf/2109.09265

Краткий обзор (ИИ): Представлена библиотека временных рядов с общей обработкой данных, калибровкой оценок и средствами оценки работы в условиях, приближённых к эксплуатации.

Зачем это в моей работе: Использую как пример другого устройства обработки порогов и жизненного цикла модели, не сводящегося к единственному partial_fit().

12. Akcay S. et al. Anomalib: A Deep Learning Library for Anomaly Detection // ICIP, 2022.

Полный текст / документация: https://arxiv.org/pdf/2202.08341

Краткий обзор (ИИ): Рассматриваются компоненты для обнаружения и локализации визуальных аномалий, экспериментов, визуализации и экспорта моделей.

Зачем это в моей работе: Нужен для проверки того, какие операции следует оставлять специфичными для изображений и нельзя подменять общим интерфейсом скоров.

13. scikit-learn. Model persistence: официальная документация.

Полный текст / документация: https://scikit-learn.org/stable/model_persistence.html

Краткий обзор (ИИ): Документация описывает способы сохранения моделей и предупреждает об опасности десериализации недоверенных pickle/joblib-файлов и проблемах переноса между версиями.

Зачем это в моей работе: Позволяет ограничить заявленные гарантии сериализации OmniAD тем, что реально проверяется.

14. PyOD. Model Save and Load: официальная документация.

Полный текст / документация: https://pyod.readthedocs.io/en/latest/model_persistence.html

Краткий обзор (ИИ): Описаны сохранение метаданных о версиях зависимостей, предупреждения о несовпадениях и подтверждение доверия перед загрузкой.

Зачем это в моей работе: Полезен для сопоставления готового механизма сохранения моделей PyOD с решением OmniAD.

15. OmniAD. Исходный код библиотеки.

Полный текст / документация: https://github.com/Ronshin-Vsevolod/OmniAD

Краткий обзор (ИИ): В репозитории находятся реализация базового детектора, семейства адаптеров, интроспекция, реестр алгоритмов, тесты и текущие средства измерений.

Зачем это в моей работе: Использую как основной материал для описания выполненной разработки. Наличие кода и наличие экспериментально доказанного преимущества — разные вещи.

16. scikit-learn. Developing scikit-learn estimators: руководство разработчика.

Полный текст / документация: https://scikit-learn.org/stable/developers/develop.html

Краткий обзор (ИИ): Описаны соглашения об инициализации оценщиков, параметрах, fit(), get_params(), set_params() и взаимодействии с инструментами scikit-learn.

Зачем это в моей работе: Нужно для содержательного сравнения собственного механизма интроспекции с уже известным программным контрактом.

17. River. HalfSpaceTrees: документация алгоритма.

Полный текст / документация: https://riverml.xyz/latest/api/anomaly/HalfSpaceTrees/

Краткий обзор (ИИ): Документация приводит назначение HalfSpaceTrees, интерфейс потокового обновления и требования к масштабу входных признаков.

Зачем это в моей работе: Нужно для проверки того, что адаптер и примеры используют алгоритм при подходящих условиях входных данных.

Анализ и выводы по обзору

Анализ построен по свойствам, существенным для OmniAD: предметная область и охват, архитектура добавления алгоритмов, самоописание API, потоковый жизненный цикл, сериализация, проверка корректности и измерения. Он основан на изученных публикациях, документации и отдельных компонентах исходного кода; количественный эксперимент по единому протоколу пока не проводился.

1. Изменение существующих решений и постановка задачи

Публикация PyOD 2019 года [8] уже показывает, что унифицированный API для обнаружения выбросов не является новой идеей. Однако считать мультидоменность PyOD неизменным свойством проекта с 2019 года было бы неверно: значительная часть нынешней поддержки текстов, изображений, временных рядов и графов появилась в апреле 2026 года, а аудио — в июне [9]. Это существенное изменение состояния предметной области. Само число доменов не может служить достаточным обоснованием OmniAD: требуется сравнивать реальные операции, способы расширения и гарантии корректности.

2. Архитектура адаптеров и расширяемость

Для работы с несколькими библиотеками OmniAD выделяет общий BaseDetector и отдельные шаблоны адаптеров для семейств бэкендов (scikit-learn, PyTorch, River, Transformers) [15]. Реализация отдельных алгоритмов за счёт этого может содержать меньше повторяющегося инфраструктурного кода. PyOD опирается на scikit-learn-совместимый контракт [9, 16] и, в частности, использует композицию кодировщика и детектора в EmbeddingOD. Эти подходы решают пересекающиеся задачи, но по-разному распределяют ответственность между общими слоями и конкретными моделями.

Достоинство подхода OmniAD пока является инженерной гипотезой: отдельные шаблоны могут облегчать интеграцию новых алгоритмов, но также вводят собственный реестр, соглашения о наследовании и поддержку дополнительного кода. Для проверки полезно взять по одному новому алгоритму из разных бэкендов и сравнить трудоёмкость подключения, набор необходимых тестов и количество специальных обходных решений. До выполнения такого сравнения утверждать, что OmniAD проще PyOD, нельзя.

3. Самоописание и границы общего контракта

В OmniAD интроспекция параметров вынесена в общий механизм, а capabilities декларируются через миксины и могут зависеть от внутреннего детектора [15]. У PyOD уже есть способы извлечения параметров благодаря соглашениям scikit-learn и описания алгоритмов в ADEngine [9, 16]. Таким образом, наличие самоописания не уникально. Подлежащие проверке свойства OmniAD — получение сведений без создания тяжёлой модели, поведение при отсутствующих необязательных зависимостях, соответствие объявленных возможностей реально работающим публичным методам.

Публикации об Anomalib и PyGOD [10, 12] подтверждают необходимость сохранять доменные особенности результатов. Например, графовые детекторы зависят от структуры графа, а

визуальные — от карт локализации. Общий API должен задавать согласованные базовые операции, но не требовать от всех алгоритмов одинакового набора специализированных методов.

4. Потоковое обучение и пороги

Работы по Half-Space Trees и River [1, 2] обосновывают поэлементное обновление моделей, но не определяют универсальный протокол для всех бэкендов. В OmniAD общий partial_fit() и ThresholdStrategy позволяют оформить пакетное и потоковое использование в одном жизненном цикле [15]. Merlion [11] представляет альтернативный подход, в котором важны также постобработка скоров и повторное обучение в сценариях эксплуатации. Выбор OmniAD оправдан для реально инкрементальных алгоритмов, но не должен навязываться моделям, поддерживающим только пакетное обучение.

При проверке программного контракта необходимо отдельно тестировать некорректные последовательности операций: например, должна ли модель изменять состояние, если обновление порога заведомо не поддерживается. Для HalfSpaceTrees требуется также проверить предположения о масштабах признаков и согласованность примеров с документацией River [17]. Эти вопросы могут привести к исправлениям кода и тестов, но сами по себе ещё не свидетельствуют о неправильности всего подхода.

5. Сериализация и воспроизводимость

Разделение состояния обёртки и нативных компонентов в OmniAD помогает не дублировать тяжёлые объекты [15]. Подход согласуется с общей идеей явного управления артефактами ML-систем [3]. При этом документирование ZIP-контейнера недостаточно для заявления о безопасной загрузке недоверенных файлов: pickle/joblib имеют известные ограничения [13]. В PyOD уже используются метаданные версий и подтверждение доверия при загрузке [14]. В текущем семестре обоснованно проверять прежде всего сохранение и восстановление модели в поддерживаемом окружении; кросс-версионную совместимость и безопасность стоит обозначить как отдельные ограничения.

6. Проверка корректности и планируемые бенчмарки

Развитый набор unit- и контрактных тестов полезен для проверки адаптеров, но не заменяет независимого эталона и корректной постановки измерений. Публикации ADBench [6] и Lones [7] важны для протоколов работы с данными и метрик качества, а работы Mytkowicz и Kalibera–Jones [4, 5] — для времени выполнения и статистической неопределённости. Поэтому задачи оценки качества детекторов и оценки дополнительных накладных расходов от слоя OmniAD следует разделять.

Проект технической спецификации новой системы бенчмаркинга пока не является утверждённой методикой. При его пересмотре необходимо проверить независимость эталонных реализаций, эквивалентность выполняемой работы, сохранение отдельных наблюдений, согласованность среды и воспроизводимость сценариев. Например, использование встроенного Python hash() для вычисления seed по строковому идентификатору не обеспечивает одинаковое значение между независимыми процессами; требуется стабильное хэширование. Для окон временного ряда необходимо учитывать не только длины массивов, но и способ привязки каждого скора ко времени и шаг окна.

Общие выводы и влияние на дальнейшую работу

Обзор не требует отказываться от выбранной темы и уже сформулированной инженерной цели: разработанные подсистемы можно описать и оценить как самостоятельный результат курсовой. Вместе с тем обоснование нужно уточнить. Новизна работы не должна связываться с первым объединением всех доменов обнаружения аномалий: существующие библиотеки решают похожие задачи, а мультидоменный PyOD быстро развился в 2026 году. Содержательный вклад OmniAD на

этом этапе — разработка конкретной архитектуры интеграции гетерогенных бэкендов и проверка её поведения.

Из анализа следуют практические уточнения задач: дополнить тестирование сценариями неуспешного потокового обновления и проверкой ограничений входных данных; при переработке спецификации бенчмарков зафиксировать воспроизводимый протокол и разделение экспериментов по качеству и производительности; не указывать преимущества по скорости, расширяемости или надёжности как доказанные до соответствующих экспериментов. Эти уточнения не требуют переписывать первоначальный список задач, но задают критерии их выполнения.

Выполнение первой задачи: обеспечение прозрачности API

Первая задача из раздела «Задачи работы» посвящена самоописанию API (пункты 1.1–1.5). В отчётах о выполненной разработке и исходном коде OmniAD описаны основные реализованные компоненты этой задачи. Ниже приведены краткое описание действий, полученные артефакты и ожидаемые последующие результаты; сравнительный эксперимент с PyOD пока не проводился.

Выполненные действия и полученные артефакты

Реализован общий механизм извлечения параметров адаптеров в omniad/utils/introspection.py. Он используется при формировании пользовательских описаний и при генерации .pyi-стабов. В публичный интерфейс добавлены omniad.describe() и omniad.list_detectors(): они позволяют получить сведения о зарегистрированном алгоритме и отфильтровать список детекторов без предварительного обучения моделей.

Создан CAPABILITY_REGISTRY, связывающий объявляемые возможности с интерфейсами-миксинами. Класс детектора сообщает о поддерживаемых операциях; для составных TF-IDF- и BERT-адаптеров доступные возможности вычисляются с учётом выбранного внутреннего детектора. Дополнительно реализованы __repr__(), __str__() и _repr_html_() для представления детектора в консоли и Jupyter.

Основные программные артефакты: omniad/utils/introspection.py; публичные функции и структуры описания в omniad/__init__.py; механизмы capabilities в omniad/core/mixins.py и omniad/core/base.py; составные адаптеры в omniad/algos/text/; соответствующие тесты интроспекции, реестра и интерфейсных контрактов в tests/. Эти пути приводятся по рабочим материалам проекта; текущие количественные показатели покрытия тестами в этом отчёте отдельно не устанавливались.

Сложности при выполнении

Задача оказалась шире извлечения сигнатуры конструктора. Описание должно работать без создания тяжёлой модели и по возможности без импорта необязательных зависимостей других алгоритмов. Для составных адаптеров возможна ситуация, когда внешний класс один и тот же, а поддерживаемые операции меняются вместе с параметром detector=. В процессе разработки был рассмотрен дополнительный миксин, зависящий от порядка наследования, но выбран более явный механизм делегирования через отдельные функции. Это уменьшает число неявных предположений об MRO.

Ожидаемые дальнейшие результаты

При расширении библиотеки потребуется поддерживать согласованность описаний, .pyi-стабов и реального поведения алгоритмов. В рамках следующих задач запланированы окончательная редакция технической спецификации новой системы бенчмаркинга, программная реализация первой воспроизводимой схемы измерений и отчётные артефакты. Сравнение стоимости добавления нового адаптера в OmniAD и PyOD остаётся возможным дополнительным экспериментом, а не уже достигнутым результатом.